# Decision trees

Forest CoverType · seven species from a contour map

LECTURE 6 | GÉRON CH. 5

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

# Where we are

The previous lecture took the error of a fitted model apart:

- Expected squared error is bias$^2$ + variance + noise, and the noise is a floor that no model gets below
- Learning curves and validation curves tell a high-bias model from a high-variance one
- A penalty — ridge, lasso, or early stopping — trades a little bias for less variance, with its strength chosen by cross-validation and the test set used once

> **THE STANDING CONSTRAINT, UNCHANGED**
>
> Split before anything is fitted. All preprocessing inside a `Pipeline` passed to cross-validation. Nothing derived from the test set in the training path. Fixed seeds. Report the fold scores, not only the mean.

Every model so far predicted with a weighted sum, $\boldsymbol{\theta}^\mathsf{T}\mathbf{x}$. Today's is not a weighted sum at all, but a sequence of yes/no questions — and the sequence *is* the explanation.

# Today

1. **The task** — a forest-cover map whose every prediction must come with a reason a person can check, and why that requirement chooses the model family before the data does
2. **The mathematics** — Gini impurity, entropy, the objective CART minimises, and how far the two criteria can disagree
3. **The method** — growing a tree, pricing the requirement, tuning inside it, reading the tree and tracing one prediction, and the per-class errors one accuracy hides

**Open Lecture 6 in Colab**

# The setting

An agency that manages a national forest has to publish a map of **forest cover type** — which species dominates each 30 by 30 metre patch of ground — over an area far too large to survey on foot.

> **THE TASK**
>
> From cartographic measurements alone — elevation, slope, aspect, distances to water, road and fire ignition points, hillshade, soil type — predict which of **seven** tree species covers the patch.

No satellite imagery, no remote sensing. Numbers a surveyor could in principle read off a contour map.

# What the output feeds

The map is not an internal artefact. It goes into the public record, and it is used to decide:

- where logging is permitted and where it is not
- which parcels qualify for habitat protection
- how fire risk is modelled for the whole district

**✗ A landowner who is refused a permit is entitled to ask *why*. So is a court.**

# The requirement that decides the model

> **STATED BY THE AGENCY, BEFORE ANY DATA IS SEEN**
>
> Every individual prediction must be accompanied by a **human-readable justification**: a statement, in terms of the measured quantities, of why *this* patch was classified as *that* species.

Note the word *individual*. Not “which variables matter in general”. Not “the model is 88% accurate”. This patch, this answer, this reason.

Requirements of this shape are ordinary in regulated work — credit, insurance, medicine, planning. They are worth meeting deliberately rather than discovering late.

# Turn the requirement into a specification

“Human-readable” is not a testable condition. Before writing any code, negotiate it into one. We asked, and this is what came back:

| Requirement | Testable form |
|---|---|
| Readable by a non-specialist | stated as conditions on the measured columns, not on transformed ones |
| Short enough to check | at most 8 conditions per prediction |
| Auditable | the same patch gives the same justification, and the rule can be applied by hand |

**✓ Eight is the number the agency chose. It is arbitrary — and it is now a hyperparameter fixed by the brief rather than by cross-validation. That is unusual and worth noticing.**

# The data

The **Forest CoverType** survey: 581,012 patches of 30 by 30 metres in the Roosevelt National Forest, Colorado, each with 54 cartographic measurements and a surveyed cover type.

| Columns | How many | Kind |
|---|---|---|
| Elevation, aspect, slope, hillshade, distances | 10 | quantitative |
| Wilderness area | 4 | already one-hot |
| Soil type | 40 | already one-hot |

No missing values, no text columns, nothing to impute. This one arrives clean — so the difficulty is entirely elsewhere.

# Split before you look — still

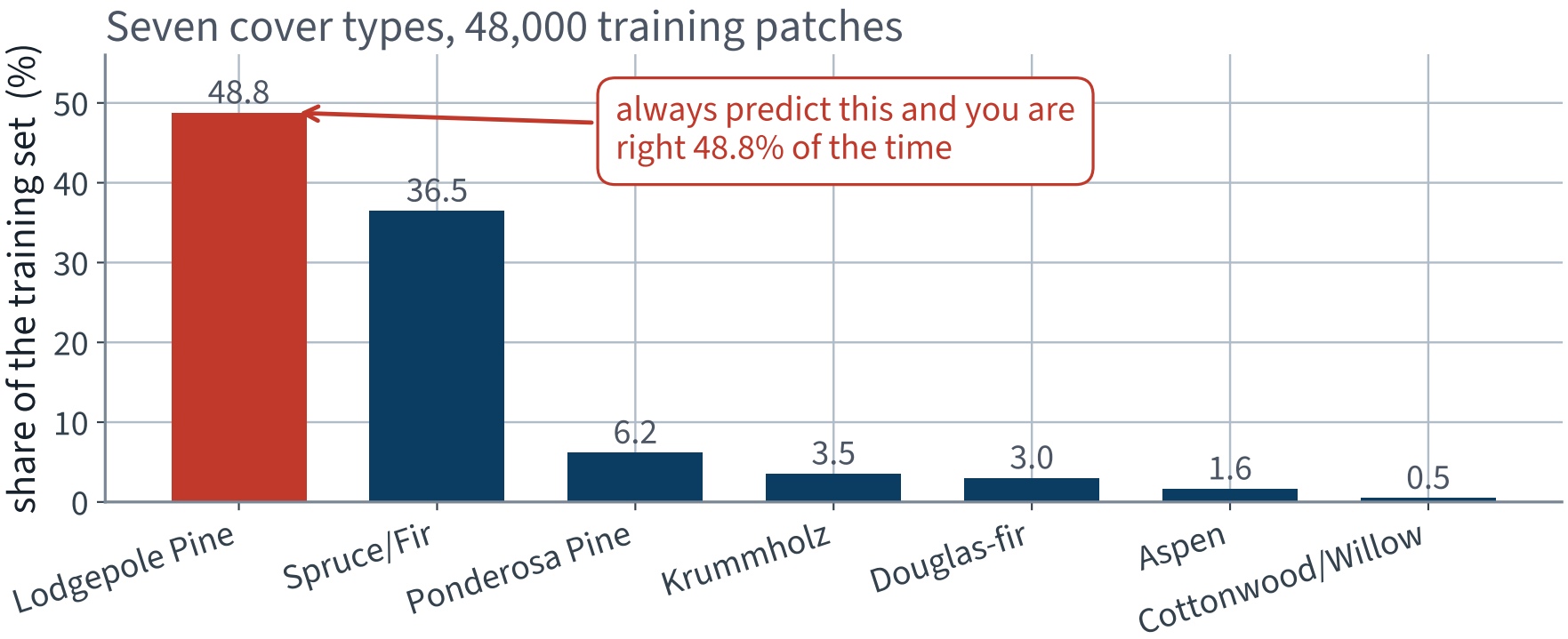
We work on a stratified subsample of 60,000 of the 581,012 patches, split 48,000 / 12,000 — a compromise for speed, and a reported one. The rule has not changed, and neither has the reason.

```
assert len(X_train) == 48_000 and len(X_test) == 12_000
assert set(X_train.index).isdisjoint(X_test.index)
assert X_train.isna().sum().sum() == 0
```

Three assertions, three seconds, and the shape of the rest of this lecture is now guaranteed rather than hoped for.

Stratifying on the label matters here: with one class at 0.5% of the data, an unstratified split can hand a fold almost none of it.

# Seven classes, and two of them are most of the forest




**✗ Two species are 85% of the ground. Whatever we build has to be judged against the model that always says “Lodgepole Pine”.**

# Metric before model, and the anchor beside it

Seven classes, no natural ordering, no asymmetric cost stated by the agency. The default is **accuracy** — reported, as always, beside the number that says what *nothing* scores.

```
from sklearn.dummy import DummyClassifier

dummy = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print(f"{dummy.score(X_test, y_test):.1%}")     # 48.8%
```

| Model | Test accuracy | Species it can ever predict |
|---|---|---|
| Always “Lodgepole Pine” | 48.8% | ✗ 1 of 7 |

**✓ Anything below 48.8% is worse than a constant. The distance above it is the only part you actually earned.**

# The question

You called `DecisionTreeClassifier().fit()`.

It chose one feature and one threshold at every node. Chose them to *minimise what*?

> **WHY YOU NEED THE ANSWER**
>
> Scikit-Learn offers two answers — `criterion="gini"` and `criterion="entropy"`. Deciding whether that switch is worth touching requires knowing what the two quantities are and how they differ, and the answer also explains why a tree ignores a rare class.

# Gini impurity

A node holds some training instances. Write $p_k$ for the fraction of them in class $k$, for $k = 1, \ldots, K$ ($K = 7$ here), and $\mathbf{p} = (p_1, \ldots, p_K)$ for the node's class distribution. A node is **pure** when some $p_k = 1$.

$$G(\mathbf{p}) \;=\; \sum_{k=1}^{K} p_k(1 - p_k) \;=\; 1 - \sum_{k=1}^{K} p_k^2$$

GINI IMPURITY — SCIKIT-LEARN'S DEFAULT

**WHAT IT MEASURES**

Draw two of the node's instances independently, with replacement. $\sum_k p_k^2$ is the probability that they are the *same* class, so $G$ is the probability that they are **different**. A pure node never disagrees with itself.

# Entropy

The same class distribution $\mathbf{p}$, scored a different way, with $0 \log_2 0 = 0$ by the limit $p \log p \to 0$:

$$H(\mathbf{p}) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

SHANNON ENTROPY, IN BITS

**WHAT IT MEASURES**

The average number of **bits** needed to transmit the class of an instance drawn from the node, under the best possible code. A pure node needs none, because the receiver already knows the answer.

# The objective CART minimises

A node of $m$ instances is split by a question $x_k \leq t_k$, which sends $m_{\mathrm{L}}$ instances left and $m_{\mathrm{R}}$ right. Its cost is the impurity $I$ — Gini or entropy — of the two children, weighted by their size:

$$J(k, t_k) \;=\; \frac{m_{\mathrm{L}}}{m}\, I_{\mathrm{L}} \;+\; \frac{m_{\mathrm{R}}}{m}\, I_{\mathrm{R}}$$

SEARCHED OVER EVERY FEATURE $K$ AND EVERY THRESHOLD $T_K$

Equivalently, maximise the **impurity decrease** $\Delta I = I_{\mathrm{parent}} - J(k, t_k)$. The parent term is the same for every candidate at this node, so the two are the same search.

# Where Gini and entropy can disagree

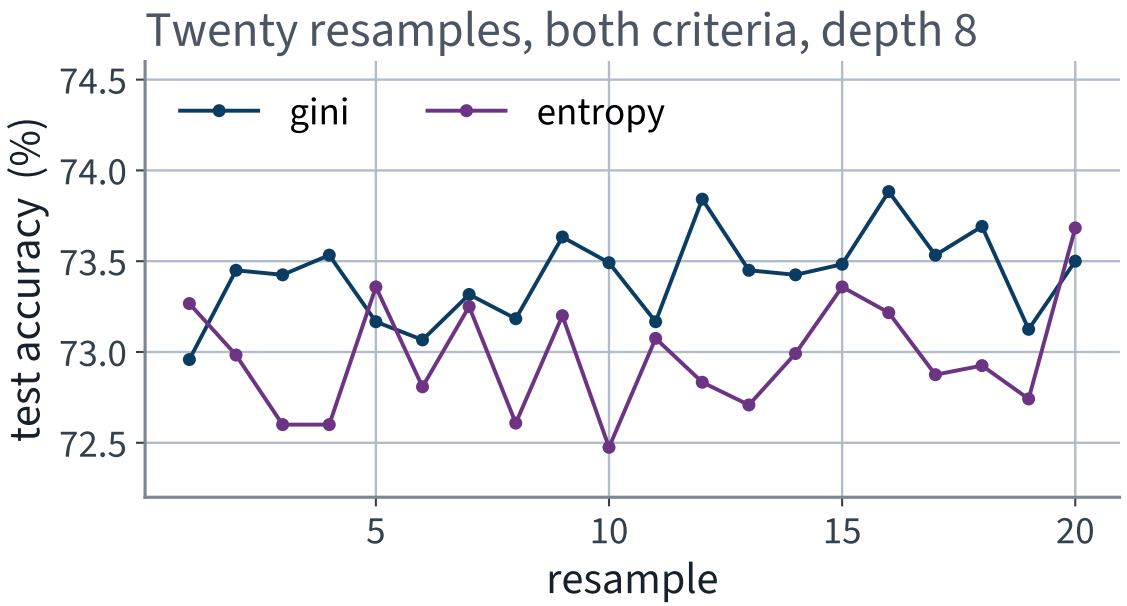
Both are zero at a pure node and grow as the classes mix, so they usually rank candidate splits the same way. How far can they part? On a two-class node with $p_1 = p$,

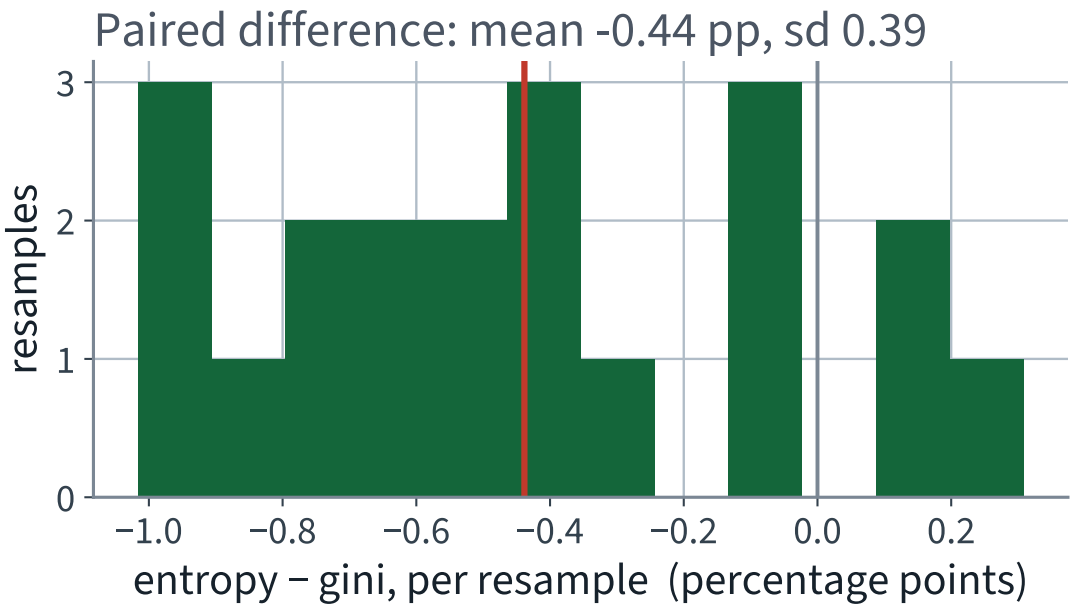
$$G(p) = 2p(1-p)$$
$$H(p) = -p\log_2 p - (1-p)\log_2(1-p)$$

$H$ has maximum 1 and $G$ has maximum $1/2$, so compare $H/2$ with $G$. Both are then $0$ at $p \in \{0, 1\}$ and $1/2$ at $p = 1/2$: they agree at three points, and the largest gap between them anywhere else is **0.055**, at $p \approx 0.10$ and again at $p \approx 0.90$.

**✓ So two candidates whose weighted impurities differ by more than 0.055 are ranked identically by both criteria. Disagreement needs a close call.**

# Does the choice matter? Twenty resamples, both criteria




Both criteria on the same twenty 80% resamples, at depth 8. Gini is above entropy on 17 of them, by under half a point: real, and not worth tuning.

# What a decision tree is

A binary tree. Each internal node asks one question about one feature; each leaf carries a prediction.

- A question is always of the form $x_k \leq t_k$ — one feature, one threshold
- Prediction: start at the root, answer the question, go left or right, repeat until a leaf
- The leaf predicts the **majority class of the training instances that reached it** — the $\arg\max_k p_k$ of that node's class distribution

| Operation | Cost | Why |
|---|---|---|
| Predict one instance | $O(\log_2 m)$ | one root-to-leaf path of a roughly balanced tree |
| Train | $O(n \times m \log_2 m)$ | at every node, sort and scan all $n$ features |

**✓ Prediction does not depend on $n$, which is why reading a prediction costs the same as making it. That is what makes the justification free.**

# Fit one, with no constraints at all

```python
from sklearn.tree import DecisionTreeClassifier

free = DecisionTreeClassifier(random_state=42)      # no depth limit
free.fit(X_train, y_train)

print(f"depth          {free.get_depth()}")
print(f"leaves         {free.get_n_leaves():,}")
print(f"train accuracy {free.score(X_train, y_train):.1%}")
```

```
depth          33
leaves         5,699
train accuracy 100.0%
```

**✗ 100.0%. A tree with 5,699 leaves and 48,000 training patches has eight patches per leaf on average. Of course it is perfect on its own data.**

# Measure it honestly

| Unconstrained tree | Accuracy |
|---|---|
| On the training set | ✗ 100.0% |
| 5-fold cross-validated, on the training set | 82.1% |
| On the test set (looked at once, at the end) | 82.6% |

82% is a real number and it is far above the 48.8% baseline, so the tree family works on this problem.

**✗ But count what a prediction rests on. The tree is 33 levels deep, and the requirement allows eight conditions.**

# What does depth actually buy?

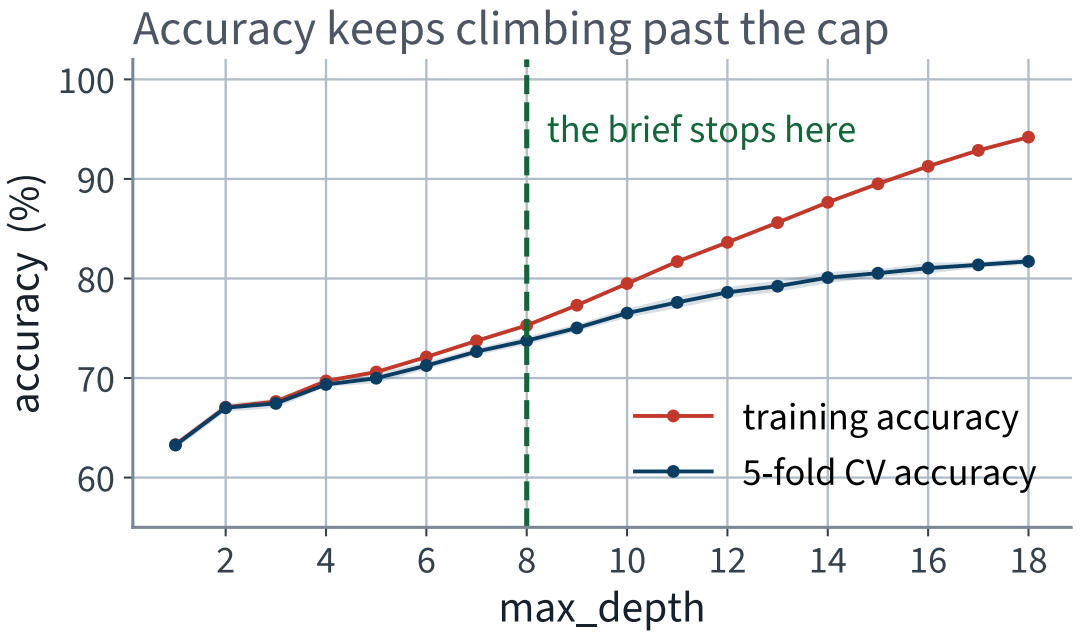
```
from sklearn.model_selection import StratifiedKFold, cross_validate

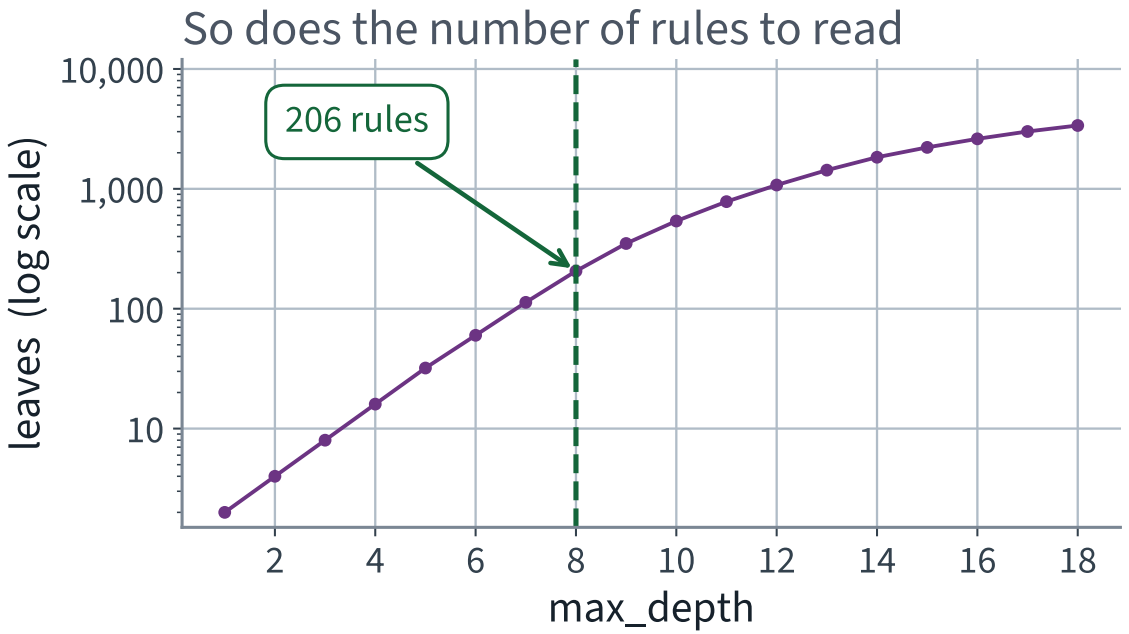
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for d in range(1, 19):
    clf = DecisionTreeClassifier(max_depth=d, random_state=42)
    scores = cross_validate(clf, X_train, y_train, cv=cv,
                            return_train_score=True, n_jobs=-1)
```

Cross-validated, on the training set. The test set is not consulted to choose anything — that rule survives from Lecture 2 and will survive to the last lecture.

# Accuracy against depth, and the price of reading it




Cross-validated accuracy is still climbing at depth 18: 8.0 points above depth 8, the price of the brief. Every one of them is bought with more rules than anyone will read.

# Tune what is left, inside the constraint

A path of depth 8 applies at most eight conditions, so the brief fixes `max_depth=8` and cross-validation gets no vote on it. What is left is `min_samples_leaf`: a split that would leave a child with fewer training instances than this is rejected.

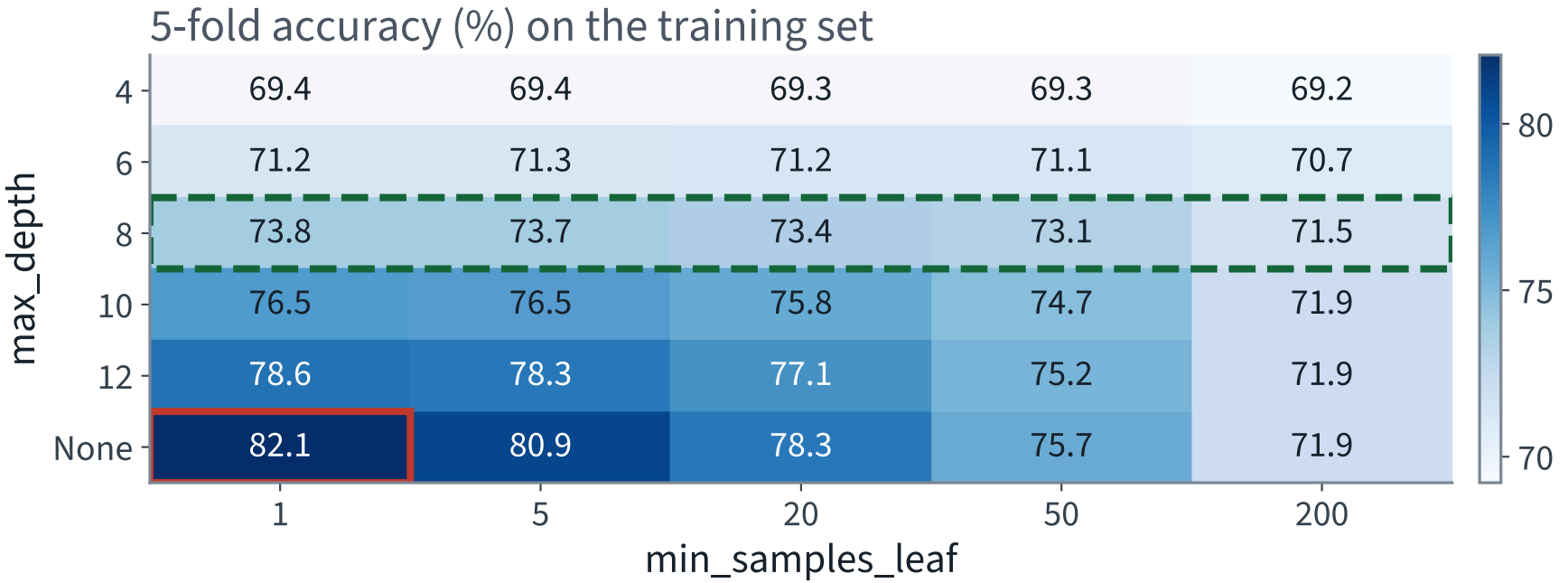
```
from sklearn.model_selection import GridSearchCV

grid = {"max_depth": [4, 6, 8, 10, 12, None],
        "min_samples_leaf": [1, 5, 20, 50, 200]}

search = GridSearchCV(DecisionTreeClassifier(random_state=42), grid,
                      cv=cv, n_jobs=-1).fit(X_train, y_train)
# 30 combinations x 5 folds = 150 fits
```

We search the whole grid anyway — including the depths we are not allowed to use — because the rows we cannot pick are exactly what tells the agency what its requirement costs.

# Thirty combinations, and only one row is available to us




The red cell is the best model. The green row is the set of models we are allowed to ship.

# Why we ship `min_samples_leaf=20`

The model states its justification as “91% of the 463 training patches in this leaf”. At `min_samples_leaf=1` that becomes *“100% of the 1”* — a single surveyed patch wearing the grammar of evidence. The brief requires a justification that can be audited; that is not one.

| | leaf | CV accuracy |
|---|---|---|
| what cross-validation prefers | 1 | 73.75% |
| what the brief requires | **20 ✓** | 73.35% |
| ✓ **price of auditability** | **—** | **0.40 points** |

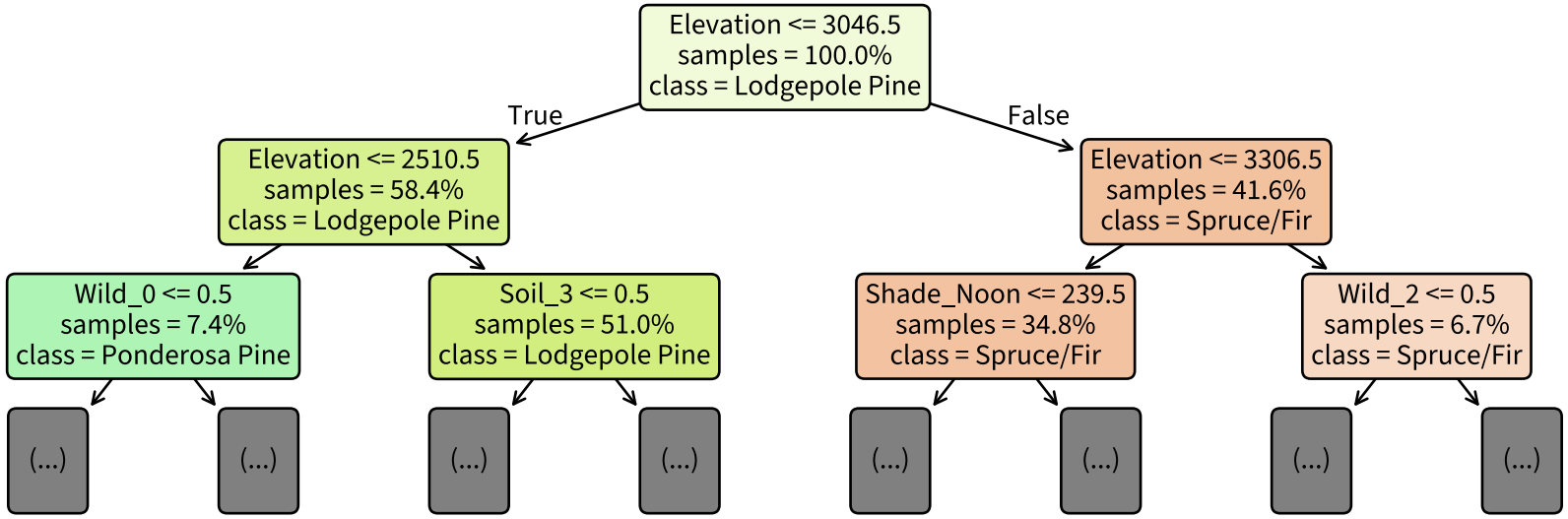
**✓ As with depth: when the brief constrains the model, the grid does not get a vote. And that 0.40 goes to the agency — a constraint that overrules a measurement has to show the measurement it overruled.**

# The model we are going to ship

```
tree = DecisionTreeClassifier(max_depth=8, min_samples_leaf=20,
                              random_state=42).fit(X_train, y_train)
```

| | Unconstrained | Depth 8, leaf 20 |
|---|---|---|
| Leaves | 5,699 | **163 ✓** |
| Columns consulted, of 54 | 47 | **24 ✓** |
| Mean conditions per justification | **✗ 17.88** | **7.97 ✓** |
| Smallest leaf | **✗ 1** | **20 ✓** |
| Training accuracy | 100.0% | 74.4% |
| Cross-validated accuracy | 82.1% | 73.4% |

# The top three levels of the model




The first three questions the model asks about any patch in Colorado. Two of them are about elevation.

# The same thing, as text

```
>>> print(export_text(tree, feature_names=names, max_depth=2, decimals=0))
|--- Elevation <= 3046
|   |--- Elevation <= 2510
|   |   |--- Wild_0 <= 0
|   |   |   |--- truncated branch of depth 6
|   |   |--- Wild_0 >  0
|   |   |   |--- truncated branch of depth 2
|   |--- Elevation >  2510
|   |   |--- Soil_3 <= 0
|   |   |   |--- truncated branch of depth 6
```

`export_text` needs no plotting library at all, and it is what you paste into an email. For a model whose selling point is that a person can read it, that matters.

# Reading one node

| Field | Meaning |
|---|---|
| `Elevation <= 3046.5` | the question; go left if true |
| `gini = 0.65` | the Gini impurity $G$ at this node — 0 would be a single species |
| `samples = 48,000` | training patches that reached this node |
| `value = [0.36, 0.49, ...]` | their class distribution $\mathbf{p}$, seven entries |
| `class = Lodgepole Pine` | the majority — what a leaf here would predict |

The threshold is 3,046.5 rather than 3,046 because CART puts the split **midway between two adjacent observed values**. Nothing in the data sits at 3,046.5 metres.

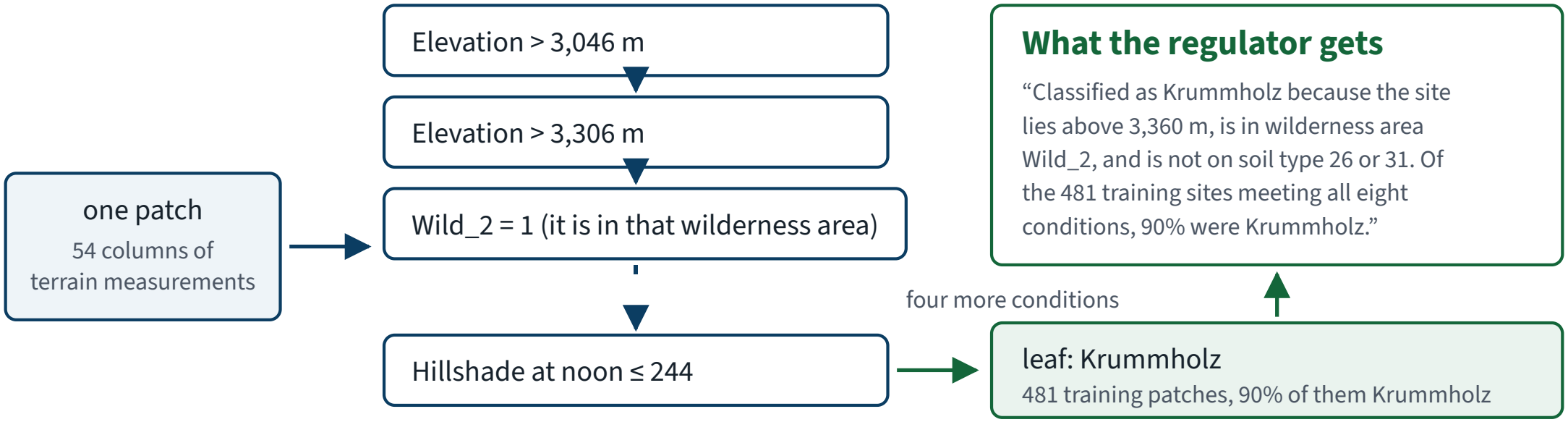
Trees need no feature scaling at all: $x_k \leq t_k$ is invariant under any strictly increasing transformation of $x_k$, so standardising a column changes its thresholds and nothing else.

# Trace one prediction, all the way down

```
def justify(tree, x, names):
    t, node, out = tree.tree_, 0, []
    while t.children_left[node] != -1:
        f, thr = t.feature[node], t.threshold[node]
        left = x[f] <= thr
        out.append((names[f], "<=" if left else ">", thr, x[f]))
        node = t.children_left[node] if left else t.children_right[node]
    return out, node
```

Nine lines, no library beyond what is already imported. The entire justification mechanism is `tree_.children_left`, `tree_.feature` and `tree_.threshold`.

# What the agency actually receives




**✓ The path *is* the justification. No extra machinery, no approximation, no second model explaining the first.**

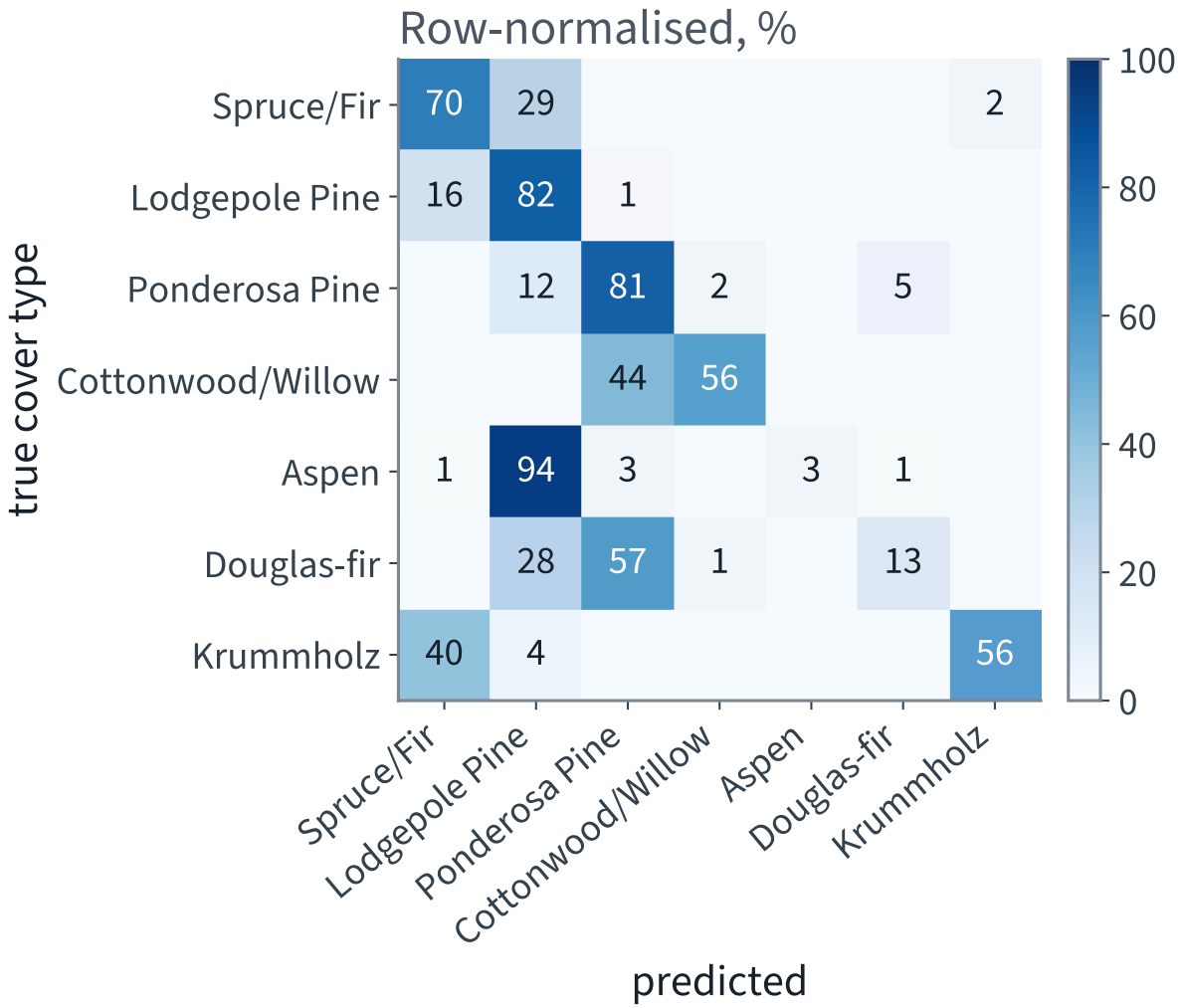
# Read the leaf carefully

91% of the 463 training patches in that leaf were Krummholz.

- That is a statement about **463 training patches**, not a probability that this patch is Krummholz
- `predict_proba` returns exactly those leaf proportions — so a tree's "probabilities" are piecewise constant and identical for every patch reaching the same leaf
- Across the 163 leaves of this tree the smallest holds 20 patches and the largest several thousand. Those deserve very different amounts of trust, and the count belongs in the justification

> **THE HONEST SENTENCE**
>
> "Of the training patches that satisfied these eight conditions, 91% were Krummholz." Not "this patch is 91% likely to be Krummholz."

# Where the errors actually are




**✗ Each row: where one true species' test patches went. Aspen's went almost entirely to Lodgepole Pine.**

# The same matrix, as one column of numbers

| Cover type | Share of the data | Recall |
|---|---|---|
| Lodgepole Pine | 48.8% | **82.0% ✓** |
| Ponderosa Pine | 6.2% | **80.8% ✓** |
| Spruce/Fir | 36.5% | 69.6% |
| Krummholz | 3.5% | 56.4% |
| Cottonwood/Willow | 0.5% | 56.1% |
| Douglas-fir | 3.0% | **✗ 12.8%** |
| Aspen | 1.6% | **✗ 2.6%** |

**✗ Overall test accuracy is 73.0%. The model finds fewer than 3 Aspen patches in every hundred, and in that one number it costs 1.6 points and is invisible.**

# What reweighting buys, and what it costs

The cause is $J$ itself: it weights each child by how many instances it holds, so a split that isolates a class at 1.6% of a node barely moves it. `class_weight="balanced"` multiplies each instance's contribution to $J$ by the inverse of its class frequency, so every class carries the same total weight.

- Aspen recall rises from 2.6% to most of the class — the split that isolates it is now worth choosing
- Overall accuracy falls sharply, because the two species that are 85% of the ground now carry 2/7 of the weight instead of 85% of it
- The notebook fits both and prints the pair. Predict the direction of each before you run it

**✓ A different model answering a different question. Which one the agency wants is not a machine learning decision — but quoting either number without the other is an argument rather than a measurement.**

# The notebook for this lecture

**Open Lecture 6 in Colab**

- **Runs on free CPU.** The download is about 11 MB
- Everything on today's slides, computed on the same rows with the same seed: the split, the baseline, the two impurity curves and the largest gap between them, the paired criterion comparison, the depth sweep, the grid, the shipped tree, the traced prediction, the confusion matrix and the reweighted tree
- Run it top to bottom first; every cell below the split depends on `X_train` and `X_test`

> **WHAT TO CHANGE BEFORE THE NEXT LECTURE**
>
> Raise `max_depth` from 8 to 12 in the cell that ships the tree, and re-run the trace. The justification grows to up to twelve conditions where the brief allows eight; the depth sweep says what that buys in accuracy, and that pair is the trade.

# What we did

1. Chose the model family from the *shape* of the answer the agency needed — a per-prediction justification in at most eight conditions — before looking at the data
2. Defined the two impurities: Gini is the probability that two draws from a node differ, entropy the bits needed to transmit a class
3. Wrote down what CART minimises — $J = \frac{m_{\mathrm{L}}}{m} I_{\mathrm{L}} + \frac{m_{\mathrm{R}}}{m} I_{\mathrm{R}}$ — and showed the two criteria part by at most 0.055 on a two-class node; measured, Gini wins 17 of 20 paired resamples by under half a point. Real, and not worth tuning
4. Grew an unconstrained tree — 100.0% on its own data, 82.1% cross-validated, 33 levels deep — and priced the eight-condition requirement at 8.0 points of accuracy
5. Shipped depth 8 with a minimum leaf of 20 against the grid's preference, because a leaf of one is not auditable, and reported the 0.40 points that cost
6. Traced one prediction to eight readable conditions and a leaf of 463 patches — a statement about training patches, not a probability
7. 73.0% overall and 2.6% recall on Aspen — a consequence of the weighted average in $J$, not a bug — so never ship the first number without the second

# In the book

| Topic | Chapter |
|---|---|
| Training, visualising and reading a decision tree | 5 |
| Gini impurity, entropy, and choosing between them | 5 |
| The CART cost function | 5 |
| Computational complexity of trees | 5 |
| Regularisation hyperparameters | 5 |
| Multiclass classification, confusion matrices, recall | 3 |
| Cross-validation and grid search | 2 |

NOT PRESENTED — FOR AFTERWARDS

# Exercises Lecture 6

five, in the style of the written paper

# Exercises — Lecture 6

**1** A node holds 100 training patches: 50 of one species, 30 of a second and 20 of a third. Compute its Gini impurity, and give the value a pure node would have. [3]

**2** On a two-class problem, entropy halved and Gini never differ by more than 0.055 at any node. Two candidate splits of the same node have weighted Gini impurities 0.30 and 0.38. Can entropy rank them the other way round? Answer, with a reason. [4]

**3** An unconstrained tree with 5,699 leaves scores 100.0% on its 48,000 training patches. Explain why that number says nothing about new patches, and name the number to report instead. [4]

Solutions on Lecture 7’s deck.

# Exercises — Lecture 6 (continued)

**4** Under the depth-8 cap the grid prefers `min_samples_leaf=1` (73.75% cross-validated) to 20 (73.35%). Give the reason to ship 20 anyway, and say what must be reported with it. [4]

**5** Per-class recall shows one class far below the rest. Give two responses that do not involve changing the model, and say which you would try first. [4]

Solutions on Lecture 7’s deck.

THE FIVE SET AT THE END OF LECTURE 5

# Solutions Lecture 5

not part of this lecture

# Solutions — Lecture 5

**1.** Write the bias–variance decomposition of the expected squared error, and name the one term no model can…

**✓ $\mathbb{E}[(y - \hat{f})^2] = \text{bias}^2 + \text{variance} + \text{Var}(y \mid x)$; the noise $\text{Var}(y \mid x)$ cannot be reduced.**

- Two passengers with identical features and different outcomes put a floor under any predictor
- A model reporting error below that floor has been measured wrongly

**2.** A learning curve shows training and validation error both high and close together, and flat as data is added.…

**✓ High bias. More data would not help.**

- The two curves have already met, so the gap — the variance — is small
- The remaining error is the model's inability to represent the function, which more rows do not change

# Solutions — Lecture 5 (2)

**3.** Ridge adds $\alpha I$ to $X^\top X$ before inverting. Give the numerical consequence, in terms of the…

## ✓ It raises every eigenvalue by $\alpha$, so the condition number falls and the inverse is stable.

- Near-collinear columns give eigenvalues near zero, which the inverse amplifies
- The bound on the solution's sensitivity improves as $\alpha$ grows — at the cost of bias

**4.** Lasso sets some weights to exactly zero; ridge shrinks every weight but sets none to zero. Explain the…

## ✓ Ridge's push towards zero fades as the weight approaches zero; lasso's keeps its full size right up to zero.

- $\partial\theta_j^2/\partial\theta_j = 2\theta_j \to 0$, so a ridge weight reaches zero only in the limit
- $\partial|\theta_j|/\partial\theta_j = \pm 1$, and at zero the subgradient is $[-1, 1]$: a data pull weaker than $\alpha$ is held at exactly zero

# Solutions — Lecture 5 (3)

**5.** You tune $\alpha$ over a grid of seventeen values by scoring each on the test set, then report the best test…

## ✓ The best-of-grid score is optimistic: it is a minimum over seventeen noisy estimates on the same rows used to choose.

- The minimum of noisy estimates is biased downward even when each one is unbiased; on the Titanic it flattered by 0.019 of log loss, on 13 splits of 20
- The honest number comes from choosing on cross-validation folds and scoring the test set once — and the bias grows with the size of the search

LECTURE 7 · GÉRON CH. 6

# Ensembles and random forests

The variance of an average of correlated predictors — one result that explains bagging, feature subsampling and random thresholds

Out-of-bag evaluation, random forests, feature importance, boosting and stacking

And what averaging costs: the justification you spent today earning

Run the Lecture 6 notebook first